SQL Injection Exploration Report

Objective:
The purpose of this exercise is to understand SQL Injection vulnerabilities by experimenting in a controlled environment using DVWA. The goal is to explore how insecure handling of user input can allow attackers to manipulate SQL queries, extract sensitive information, and bypass access controls.

Methods Used:
1. **Basic Injections:** I began testing the input field in the SQL Injection section of DVWA with simple inputs that revealed how the system processed queries.
2. **Union-Based Exploration:** I experimented with combining queries to reveal additional information from the database, such as usernames and password hashes.
3. **Advanced Techniques:** At higher DVWA security levels, I investigated input validation mechanisms and experimented with bypass techniques such as alternate syntax, mixed-case keywords, and obfuscated comments.

Results:
During the exploration, I observed how different payloads produced varied results. In the Low security level, I was able to extract user credentials directly from the database. At Medium and High levels, input validation made direct injection more difficult, but alternate forms of SQL keywords and different commenting styles enabled further testing.

Screenshot Placeholder: (Insert screenshot of successful query execution here)

Screenshot Placeholder: (Insert screenshot of extracted data here)

Security Recommendations:
To mitigate SQL Injection vulnerabilities, the following practices are recommended:
- Use parameterized queries (prepared statements) instead of direct string concatenation.
- Validate and sanitize user input using whitelisting techniques.
- Implement the principle of least privilege on database accounts.
- Employ Web Application Firewalls (WAFs) to detect and block malicious patterns.

Discussion:
This exercise demonstrates how SQL Injection can compromise sensitive data if user input is not properly handled. While basic injections expose weaknesses quickly, advanced bypass techniques show that simple filtering is insufficient. In real-world applications, such vulnerabilities could lead to credential theft, unauthorized access, or full database compromise.

End of Report